技 术 方 案 书

altium-mcp：AI 辅助原理图设计系统（MCP 桥接服务器）

编制：连子恒

编制依据：技术方案书三件套（全局架构 / 模块依赖 / 业务流程）

评审：2026-09-16 — 09-17　评审通过

版本：V1.0（9 月版）

阶段一：全局架构与核心设计思路

1. 核心功能与根本问题

一句话概括：altium-mcp 是一个基于 Model Context Protocol (MCP) 的桥接服务器，通过 JSON 文件通信机制将 AI 助手（Trae 等）与 Altium Designer 连接。当前处于 **AI 顾问模式（Advisory Mode）**，仅提供读取、分析、审查能力，所有写入操作通过 Feature Flag 默认禁用，解决"AI 无法直接驱动专业 EDA 工具"的根本问题。

2. 架构模式与选择原因

采用的架构模式：分层 + 桥接代理 (Layered + Bridge Agent)

项目采用**分层架构**结合**文件桥接代理模式**，而非传统的微服务或事件驱动架构。选择该模式的原因：

因素	说明
Altium 约束	Altium Designer 仅支持通过 DelphiScript 脚本自动化，无 HTTP/RPC 接口，必须使用文件作为进程间通信媒介

因素	说明
进程隔离	MCP 服务器运行在 Node.js 进程中，Altium 运行在独立 GUI 进程中，文件桥接天然实现解耦
简单性	文件读写比 socket/管道更简单，无需处理连接管理、序列化协议栈
MCP 标准兼容	MCP 协议基于 stdio，服务器作为 MCP 客户端与 Altium 之间的翻译层，职责清晰

架构选择的核心权衡

传统做法会考虑 socket 或 named pipe 实现进程间通信，但 Altium 的 DelphiScript 环境对网络 API 支持有限，且调试困难。JSON 文件方案虽然牺牲了性能（轮询 500ms），但换来了**零依赖、易调试、崩溃隔离**三大优势——任何一方崩溃都不会影响另一方，且可通过直接查看 `request.json/response.json` 排查问题。

3. 分层结构与职责边界

项目分为四层，每层有明确的职责与边界：

3.1 接口层 (MCP Interface Layer)

文件：`src/index.ts`、`src/toolDefinitions.ts`、`src/mcpInstructions.ts`

职责	边界
注册 MCP 工具并定义输入 Schema	不包含业务逻辑，仅做参数校验和转发
向 MCP 客户端发送 `instructions` 指引	不直接操作文件系统（除 `get_server_status` 诊断）
将工具调用参数转换为 bridge command	不关心 Altium 如何执行命令

关键类/函数：

- `McpServer` 实例：注册 22 个活动工具（只读 + 库 + 输出 + 实用），通过 `StdioServerTransport` 与客户端通信
- `registerLiveCommand()`：工厂函数，统一注册只读 PCB 查询工具
- `jsonResult()` / `errResult()`：统一 MCP 响应格式封装
- Feature Flag 常量：`ENABLE_SCHEMATIC_WRITES`、`ENABLE_PCB_WRITES`、`ENABLE_SYMBOL_CREATION`（均默认 `false`），控制 7 个写入工具的注册与执行

3.2 桥接层 (Bridge Layer)

文件： `src/bridge/altiumBridge.ts`、`src/bridge/lock.ts`、`src/workspaceLayout.ts`、`src/altiumPaths.ts`、`src/altiumBundleFingerprint.ts`

职责	边界
写入 `request.json`，启动 Altium 进程执行脚本	不解析 bridge 响应的业务语义
轮询读取 `response.json` 并做 Schema 校验	不直接调用 Altium API
文件锁序列化并发调用	不管理 Altium 生命周期（仅启动，不关闭）
脚本包同步与指纹管理	不修改脚本内容

关键类/函数：

- `AltiumBridge` 类：核心桥接器，`executeCommand()` 是唯一入口
- `withBridgeLock()`：基于文件锁的互斥控制，支持 stale lock 检测
- `ensureAltiumScriptBundle()`：基于 SHA256 指纹的增量同步
- `launchAltiumRunScript()`：通过 `-RScriptingSystem:RunScript` 参数启动 Altium

3.3 工具/校验层 (Tools & Validation Layer)

文件： `src/tools/editSchematic.ts`、`src/tools/getSchematicData.ts`、`src/tools/designators.ts`、`src/tools/helpers.ts`

职责	边界
使用 Zod 定义工具输入 Schema 并校验	不执行 bridge 命令（交由接口层）
对 bridge 返回数据做后处理（如提取 designators）	不直接与 Altium 交互
参数语义校验（如 `parameter_names` 和 `parameter_values` 长度一致）	不处理 Altium 侧的业务逻辑

关键类/函数：

- `editSchematicInputSchema`：使用 `superRefine` 实现 11 种 action 的差异化校验
- `getSchematicDataInputSchema`：定义 `include_queries` 枚举（15 种过滤桶）
- `parseDesignatorsFromComponentData()`：从组件数据中提取位号列表

3.4 执行层 (DelphiScript Execution Layer)

文件： `altium-scripts/Altium_API.pas`、`schematic_utils.pas`、`schematic_edit.pas`、`pcb_utils.pas`、`other_utils.pas`、`json_utils.pas`、`pcb_layout_duplicator.pas`

职责	边界
读取 `request.json`，解析命令和参数	不与 MCP 协议交互
调用 Altium API 执行 PCB/原理图操作	不写 `request.json`（只读）
序列化结果为 JSON 并写入 `response.json`	不管理文件锁（由 Node 侧控制）
错误日志写入 `bridge_last_error.txt`	不处理超时（由 Node 侧轮询控制）

关键类/函数：

- `Run` 过程（`Altium_API.pas`）：DelphiScript 唯一入口，读取请求并分发命令
- `ExecuteCommand()`：命令路由器，通过 `case` 分发到 20+ 个命令处理器，写入命令受 Feature Flag 常量门控
- `WriteResponse()`：统一响应封装，自动检测 `ERROR:` 前缀并设置 `success` 字段
- `GetSchematicData()`：原理图数据采集，支持 legacy 和 filtered 两种模式

- `SchematicEditMain()`：原理图编辑入口，分发 11 种 action（当前通过 `ENABLE_SCHEMATIC_WRITES` 禁用）

4. 关键设计模式

4.1 命令模式 (Command Pattern)

应用场景：MCP 工具调用与 DelphiScript 命令的映射。

每个 MCP 工具对应一个 bridge command 字符串（如 `get_pcb_layers`、`schematic_edit`），`AltiumBridge.executeCommand()` 将命令和参数写入 `request.json`，DelphiScript 侧的 `ExecuteCommand()` 通过 `case` 语句路由到具体处理器。命令对象（request.json）封装了执行所需的所有信息，调用者无需知道接收方如何处理。

4.2 策略模式 (Strategy Pattern)

应用场景：`edit_schematic` 工具的多 action 分发。

`editSchematicInputSchema` 通过 `editSchematicActionEnum` 定义 11 种策略（`set_component_transform`、`place_component`、`add_wire` 等），每种策略有不同的参数要求和校验规则。`SchematicEditMain` 根据 `action` 字段选择对应的执行策略，新增 action 只需添加枚举值和对应分支，不修改现有逻辑。

4.3 代理模式 (Proxy Pattern)

应用场景：`AltiumBridge` 作为 Altium Designer 的代理。

Node 侧无法直接调用 Altium API，`AltiumBridge` 通过文件通信代理所有对 Altium 的操作。调用方（MCP 工具）像调用本地方法一样调用 `bridge.executeCommand()`，代理内部处理了进程启动、文件读写、锁管理、超时控制等横切关注点。

4.4 模板方法模式 (Template Method)

应用场景：DelphiScript 中所有数据获取函数遵循统一的执行模板。

`GetAllNets`、`GetPCBLayers`、`GetPCBRules` 等函数都遵循相同的流程：获取 Board → 创建迭代器 → 遍历对象收集属性 → `BuildJSONObject` → `WriteJSONToFile` → 返回 JSON 字符串。虽然 DelphiScript 不支持继承，但通过约定一致的代码结构实现了模板方法的效果。

4.5 工厂模式 (Factory Pattern)

应用场景： `registerLiveCommand()` 函数和 DelphiScript 的 `SchObjectFactory`。

Node 侧用 `registerLiveCommand()` 工厂函数批量注册结构相似的只读 PCB 工具，避免重复代码。DelphiScript 侧用 `SchObjectFactory(eSchComponent)` 等调用按枚举类型创建原理图对象，统一了对象的创建接口。

5. 核心数据流转路径

```
flowchart TD
    A["MCP Client<br/>(Trae)"] -->|"stdio JSON-RPC"|
        B["McpServer<br/>src/index.ts"]
    B -->|"工具调用 + 参数"| C["Zod Schema 校验<br/>src/tools/*.ts"]
    C -->|"校验通过"|
        D["AltiumBridge.executeCommand<br/>src/bridge/altiumBridge.ts"]
    C -->|"校验失败"| C1["返回错误 JSON"]
    D -->|"获取文件锁"| E["withBridgeLock<br/>src/bridge/lock.ts"]
    E -->|"锁获取成功"| F["写入 request.json<br/>{protocolVersion, command,
        ...params}"]
    E -->|"锁超时"| E1["返回 LockTimeoutError"]
    F -->|"spawn X2.EXE"| G["Altium Designer
        进程启动<br/>-RScriptingSystem:RunScript"]
    G -->|"加载 Altium_API.PrjScr"| H["DelphiScript Run
        过程<br/>altium-scripts/Altium_API.pas"]
    H -->|"读取 request.json"| I["ExecuteCommand 路由<br/>case CommandName of"]
    I -->|"PCB 命令"| J["pcb_utils.pas<br/>GetPCBLayers/GetAllNets/..."]
    I -->|"原理图读取"| K["schematic_utils.pas<br/>GetSchematicData"]
    I -->|"原理图编辑"| L["schematic_edit.pas<br/>SchematicEditMain"]
    I -->|"工作/其他"| M["other_utils.pas<br/>GetWorkspaceOverview"]
    J -->|"Altium PCB API"| N["IPCB_Board 对象模型"]
    K -->|"Altium SCH API"| O["ISch_Document 对象模型"]
    L -->|"Altium SCH API"| O
    M -->|"Altium Workspace API"| P["IWorkspace 对象模型"]
    N -->|"数据收集"| Q["json_utils.pas<br/>BuildJSONObject + WriteJSONToFile"]
    O -->|"数据收集"| Q
    P -->|"数据收集"| Q
    Q -->|"写入 response.json"| R["workspace/response.json"]
    D -->|"500ms 轮询"| S{"response.json<br/>存在?"}
    S -->|"是"| T["Zod Schema 校验响应"]
```

```
    S -->|"否,未超时"| S
    S -->|"超时"| S1["返回 TIMEOUT"]
    T -->|"校验通过"| U["augmentBridgeFailure<br/>合并 bridge_last_error.txt"]
    U -->|"返回 BridgeResponse"| V["jsonResult 封装"]
    V -->|"stdio JSON-RPC"| A
    R -->|"读取后删除"| W["清理 response.json"]
```

数据形态变化

节点	数据形态
MCP Client → Server	JSON-RPC over stdio，含工具名和参数对象
Schema 校验	Zod 解析后的类型安全对象
request.json	`{ "protocolVersion": 1, "command": "xxx", ...params }`
DelphiScript 内部	`TStringList` 逐行解析的键值对
Altium API 返回	IPCB_Board / ISch_Document 等 COM 对象
response.json	`{ "success": boolean, "result"?: any, "error"?: string }`
MCP Server → Client	`{ "content": [{ "type": "text", "text": "JSON string" }] }`

关键设计点

文件锁序列化：`.bridge.lock` 文件确保同一时刻只有一个 DelphiScript 实例在运行，避免 Altium 多实例冲突。锁文件写入 PID，`removeStaleBridgeLock()` 通过 `process.kill(pid, 0)` 检测进程是否存活，自动清理崩溃残留的锁。

指纹同步机制：`altiumBundleFingerprint.ts` 计算整个 `altium-scripts/` 目录的 SHA256 指纹，只有内容变化时才同步到 workspace，避免每次启动都复制文件。同步采用镜像策略（增删改），而非删除重建，保持 Altium 对脚本项目路径的稳定引用。

错误增强：`augmentBridgeFailure()` 在 bridge 返回失败时，读取 Altium 写入的 `bridge_last_error.txt` 并合并到错误信息中，使 AI 能看到 DelphiScript 侧的详细错误。对于编译期错误（Altium 不发送给 bridge），设计了

bridge_user_reported_error.txt 机制，用户手动粘贴 Altium Messages 面板内容后由 get_server_status 读取。

5. Feature Flag 门控架构

设计动机

在项目开发过程中发现，通过 MCP 桥接对 Altium 原理图进行写入操作（放置元件、连线等）存在坐标变换、元件拉伸等不可控问题。为保证 AI 顾问模式的稳定性，采用 Feature Flag 机制将所有写入操作从工具列表中隐藏，改为纯读取 + 分析 + 建议的 Advisory Mode。

Feature Flag 定义

常量	默认值	控制的工具	定义位置
ENABLE_SCHEMATIC_WRITES	false	edit_schematic（11 种 action）	src/index.ts + Altium_API.pas
ENABLE_PCB_WRITES	false	set_component_position, move_components, create_net_class, set_pcb_layer_visibility, layout_duplicator_apply	src/index.ts + Altium_API.pas
ENABLE_SYMBOL_CREATION	false	create_schematic_symbol	src/index.ts + Altium_API.pas

三层门控机制

```
调用链路: MCP Client → src/index.ts → Altium_API.pas


Layer 1: 工具注控 (src/index.ts)
├─ if (ENABLE_SCHEMATIC_WRITES) { server.registerTool("edit_schematic", ...) }
├─ if (ENABLE_PCB_WRITES) { server.registerTool("set_component_position", ...)
    }
└─ if (ENABLE_SYMBOL_CREATION) {
    server.registerTool("create_schematic_symbol", ...) }
→ 关闭时工具不出现在 MCP tools/list 中，客户端无法发现


Layer 2: 执行命令门控 (Altium_API.pas ExecuteCommand)
├─ if (not ENABLE_SCHEMATIC_WRITES) and (Command='schematic_edit') → 返回
```

```
    WRITE_OPERATIONS_DISABLED
├─ if (not ENABLE_PCB_WRITES) and (Command in [...]) → 返回
    WRITE_OPERATIONS_DISABLED
└─ if (not ENABLE_SYMBOL_CREATION) and (Command='create_schematic_symbol') →
    返回 WRITE_OPERATIONS_DISABLED
→ 即使绕过 Layer 1, DelphiScript 侧也会拦截


Layer 3: 操作级门控 (SchematicEditMain 等)
└─ 写入操作内部的具体函数调用同样检查 Feature Flag 常量
→ 防止未来重构时遗漏
```

工具分类（当前状态）

分类	工具数	状态	工具列表
环境诊断	3	✅ 启用	get_server_status, configure_altium_exe, altium_ping
工作区	1	✅ 启用	get_workspace_projects
PCB 只读	7	✅ 启用	get_pcb_layers, get_pcb_rules, get_all_nets, get_pcb_layer_stackup, get_all_designators, get_component_pins, get_selected_components
原理图读取	1	✅ 启用	get_schematic_data
库与记忆	5	✅ 启用	get_library_symbol_reference, search_library_symbol, import_library_components, get_memory_status, refresh_memory_cache
布局发现	1	✅ 启用	layout_duplicator
输出作业	2	✅ 启用	get_output_job_containers, run_output_jobs
截图与本地	2	✅ 启用	take_view_screenshot, file_mode_capabilities

分类	工具数	状态	工具列表
原理图写入	**1**	❌ **禁用**	**edit_schematic** (11 actions)
PCB 写入	**5**	❌ **禁用**	**set_component_position, move_components, create_net_class, set_pcb_layer_visibility, layout_duplicator_apply**
符号创建	**1**	❌ **禁用**	**create_schematic_symbol**

恢复写入操作

需要恢复某类写入操作时，需同步修改两处：

1. `src/index.ts`：将对应常量改为 `true`
2. `altium-scripts/Altium_API.pas`：将对应常量改为 `True`
3. 重新编译 TypeScript（`npx tsc`）+ 在 Altium 中重新编译脚本项目

阶段一完成。 以上分析基于 `src/` 目录下全部 TypeScript 源码、`altium-scripts/` 下全部 DelphiScript 源码、`schemas/` 下的 JSON Schema 以及 `package.json` 配置文件的真实代码内容。

请确认或补充信息，确认后我将进行阶段二（模块依赖与文件调用关系）的分析。

阶段二：模块依赖与文件调用关系

1. 核心模块划分与职责

项目分为两大子系统：**Node.js MCP 服务器**（TypeScript）和 **DelphiScript 执行层**（Pascal）。每个子系统进一步划分为多个模块。

1.1 Node.js 侧模块

模块	文件	职责	对外暴露接口
MCP Server	`src/index.ts`	MCP 工具注册、请求转发、响应封装、Feature Flag 门控	22 个活动 MCP 工具（含 7 个写入工具的 Feature Flag 门控逻辑）
Tool Definitions	`src/toolDefinitions.ts`	工具描述字符串、只读/编辑/配置注解	`DESCRIPTION_*` 常量、`annotations*` 注解对象
MCP Instructions	`src/mcpInstructions.ts`	服务器初始化时发送给客户端的指引文本	`ALTIUM_MCP_INSTRUCTIONS` 常量字符串
Bridge	`src/bridge/altiumBridge.ts`	文件桥接通信、Altium 进程启动、响应轮询	`AltiumBridge` 类、`launchAltiumRunScript()`、`readBridgeDiagnosticSnippets()`
Lock	`src/bridge/lock.ts`	基于文件锁的并发控制	`withBridgeLock()`、`LockTimeoutError` 类
Workspace Layout	`src/workspaceLayout.ts`	脚本包同步、工作区目录管理	`ensureAltiumScriptBundle()`、`scriptProjectPath()`
Altium Paths	`src/altiumPaths.ts`	Altium 安装路径自动发现	`discoverAltiumExe()`
Bundle Fingerprint	`src/altiumBundleFingerprint.ts`	脚本目录的 SHA256 指纹计算	`computeBundledAltiumScriptsFingerprint()`、`listBundledScriptAbsPathsSorted()`
Config	`src/config.ts`	用户配置加载/保存/合并	`loadUserConfig()`、`saveUserConfig()`、`resolveWorkspaceRoot()`、`resolveAltiumExe()`

模块	文件	职责	对外暴露接口
Edit Schematic	`src/tools/editSchematic.ts`	原理图编辑工具的 Zod Schema 校验	`editSchematicInputSchema`、`EditSchematicInput` 类型
Get Schematic Data	`src/tools/getSchematicData.ts`	原理图数据查询工具的 Zod Schema 校验	`getSchematicDataInputSchema`、`GetSchematicDataInput` 类型
Designators	`src/tools/designators.ts`	从组件数据中提取位号列表	`parseDesignatorsFromComponentData()`
Helpers	`src/tools/helpers.ts`	MCP 响应格式封装	`jsonResult()`、`errResult()`
File Mode	`src/fileMode.ts`	离线文件模式能力声明（占位）	`getFileModeCapabilities()`

1.2 DelphiScript 侧模块

模块	文件	职责	对外暴露接口
Entry	`Altium_API.pas`	请求读取、命令路由、响应写入	`Run` 过程（唯一入口）、`ExecuteCommand()`、`WriteResponse()`

模块	文件	职责	对外暴露接口
PCB Utils	pcb_utils.pas	PCB 数据读取与操作	GetAllNets()、GetPCBLayers()、GetPCBRules()、GetPCBLayerStackup()、GetAllComponentData()、GetComponentPinsFromList()、SetComponentPosition()（Feature Flag 禁用）、MoveComponentsByDesignators()（Feature Flag 禁用）
Schematic Utils	schematic_utils.pas	原理图数据读取与符号库操作	GetSchematicData()、GetLibrarySymbolReference()、CreateSchematicSymbol()（Feature Flag 禁用）、SearchLibrarySymbol()、McpParseJSONLineValue()
Schematic Edit	schematic_edit.pas	原理图编辑操作（当前通过 Feature Flag 整体禁用）	SchematicEditMain()（11 种 action 分发，受 ENABLE_SCHEMATIC_WRITES 门控）

模块	文件	职责	对外暴露接口
Json Utils	`json_utils.pas`	JSON 序列化/反序列化工具	`BuildJSONObject()`、`BuildJSONArray()`、`WriteJSONToFile()`、`AddJSONProperty/Number/Integer/Boolean()`、`JSONEscapeString()`、`TrimJSON()`、`JSONPairStr()`
Other Utils	`other_utils.pas`	工作区管理、文档聚焦、截图、OutJob	`GetWorkspaceOverview()`、`EnsureDocumentFocused()`、`GetOutputJobContainers()`、`RunOutputJobs()`
Layout Duplicator	`pcb_layout_duplicator.pas`	PCB 布局复制与网表追踪	`GetLayoutDuplicatorComponents()`（只读，启用）、`ApplyLayoutDuplicator()`（Feature Flag 禁用）

2. 模块依赖关系

2.1 Node.js 侧依赖树

```
src/index.ts
├── src/bridge/altiumBridge.ts              (显式 import)
│   ├── src/altiumPaths.ts                  (显式 import)
│   ├── src/config.ts                       (显式 import)
│   ├── src/workspaceLayout.ts              (显式 import)
│   │   ├── src/altiumBundleFingerprint.ts  (显式 import)
│   │   └── ── (无外部依)
│   └── src/bridge/lock.ts                  (显式 import)
│       └── ── (仅依赖 node:fs, node:path)
├── src/fileMode.ts                         (显式 import)
├── src/mcpInstructions.ts                  (显式 import)
├── src/toolDefinitions.ts                  (显式 import)
```

```
├── src/tools/editSchematic.ts              (显式 import)
│   └── zod                                 (依赖)
├── src/tools/getSchematicData.ts           (显式 import)
│   └── zod                                 (依赖)
├── src/tools/designators.ts                (显式 import)
├── src/tools/helpers.ts                    (显式 import)
└── @modelcontextprotocol/sdk               (npm 依赖)
    └── zod                                 (npm 依赖)
```

说明：Node.js 侧所有依赖均为**显式 import/export**，无隐式依赖（反射、动态加载等）。依赖方向为单向：`index.ts` → `bridge/` + `tools/`，`bridge/` → `config.ts` + `workspaceLayout.ts` + `lock.ts`，无循环依赖。

2.2 DelphiScript 侧依赖树

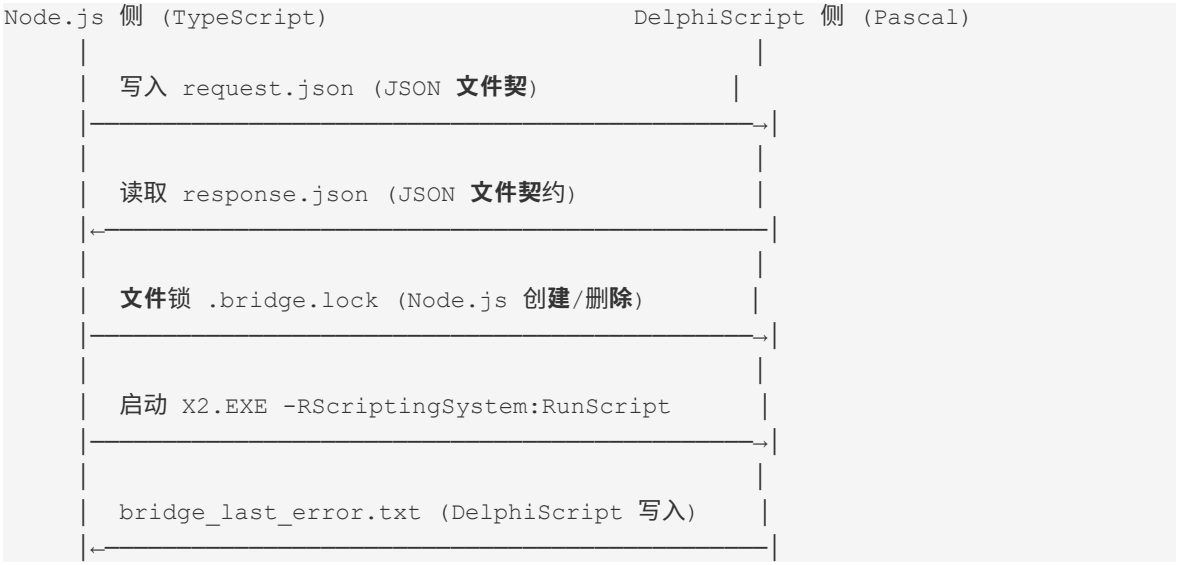
```
Altium_API.pas
├── (运行时加载) pcb_utils.pas
│   └── json_utils.pas                      (依赖，所有 JSON 函数)
├── (运行时加载) schematic_utils.pas
│   └── json_utils.pas                      (依赖)
├── (运行时加载) schematic_edit.pas
│   ├── json_utils.pas                      (依赖)
│   └── schematic_utils.pas                 (依赖: McpParseJSONLineValue,
    McpNormalizeWindowsPath)
├── (运行时加载) other_utils.pas
│   └── json_utils.pas                      (依赖)
├── (运行时加载) pcb_layout_duplicator.pas
│   └── json_utils.pas                      (依赖)
└── (运行时加载) json_utils.pas            (基础工具，无任何 .pas 依赖)
```

说明：DelphiScript 使用**脚本项目加载顺序**而非 `uses` 语句管理依赖。`Altium_API.PrjScr` 的项目文件中按顺序列出所有 `.pas` 文件，加载顺序决定了符号可见性。`json_utils.pas` 是唯一的**基础层**，被所有其他模块依赖，自身不依赖任何项目内 `.pas` 文件。

Altium 运行时隐式依赖（所有 `.pas` 文件共享）：

- `GetWorkspace`、`SchServer`、`PCBServer`、`Client`（Altium 全局对象）
- `CoordToMils`、`MilsToCoord`（坐标转换）
- `MkSet`、`Point`、`TStringList` 等 Delphi 运行时函数
- `ISch_Document`、`IPCB_Board`、`IWorkspace` 等接口类型
- `eSchComponent`、`eWire`、`ePadObject` 等枚举常量

2.3 跨语言依赖（隐式）



说明：跨语言依赖通过**文件系统契约**实现，这是一种隐式耦合。`request.json` 的格式由 `bridge-request.schema.json` 定义，`response.json` 的格式由 `bridge-response.schema.json` 定义。双方独立实现各自的读写逻辑，任何一方修改了协议格式都会导致通信失败。

3. 高风险依赖节点

3.1 高扇出模块（被 ⩾5 处引用）

模块	文件	扇出数	被谁引用
Json Utils	`json_utils.pas`	6	`Altium_API.pas`、`pcb_utils.pas`、`schematic_utils.pas`、`schematic_edit.pas`、`other_utils.pas`、`pcb_layout_duplicator.pas`

模块	文件	扇出数	被谁引用
AltiumBridge	src/bridge/altiumBridge.ts	5	src/index.ts（直接）、src/index.ts 通过 bridge 实例被 12 个工具调用

风险分析：

- **`json_utils.pas`** 是所有 DelphiScript 模块的**唯一 JSON 工具**，如果 JSON 序列化逻辑存在 bug（如 FloatToStr 在不同语言环境下的逗号/点号问题），会导致所有模块的响应格式异常。当前代码已通过 StringReplace(FloatToStr(Value), ',', '.', REPLACEALL) 部分规避了此问题，但 TStringList 在 SaveToFile / LoadFromFile 过程中可能引入换行符不一致的问题。
- **`AltiumBridge`** 是 MCP 服务器到 Altium 的**唯一通道**，所有工具的执行都经过 executeCommand()。该类的任何性能问题（如锁等待时间过长）、超时配置不当、或文件 I/O 异常都会导致全局性故障。当前设计已通过 edit_schematic 单独设置 240s 超时（默认 120s）来部分缓解此问题。

3.2 循环依赖

未发现循环依赖。Node.js 侧依赖方向为单向 index.ts → bridge/ → config/workspace，无反向引用。DelphiScript 侧依赖树为有向无环图，json_utils.pas 为最底层。

3.3 跨层调用

未发现严重的跨层调用。所有调用遵循分层边界：

- MCP 工具不直接调用 Altium API（通过 Bridge 代理）
- Bridge 不直接处理业务逻辑（仅负责文件通信和进程管理）
- DelphiScript 不直接与 MCP 协议交互（通过文件解析/写入）

潜在风险点：

风险类型	位置	说明
代码重复	`schematic_utils.pas` 与 `schematic_edit.pas`	两者各自实现了 `SchPathsEqual` / `SchEditPathsEqual`（路径比较）、`SchDataParseStringArray` / `SchEditParseStringArray`（JSON 数组解析）。重复代码约 30 行，增加了维护成本，修改一处需同步修改另一处
加载顺序耦合	`Altium_API.PrjScr`	DelphiScript 没有 `uses` 语句，依赖项目文件中的文件加载顺序。如果 `schematic_edit.pas` 在 `schematic_utils.pas` 之前加载，前者的 `McpParseJSONLineValue` 调用会失败
硬编码路径	`altium-mcp-main/` 下的旧版 `Altium_API.pas`	`altium-mcp-main/server/AltiumScript/Altium_API.pas` 使用硬编码路径 `C:\Users\Public\altium_mcp\`，与当前版本的工作区路径机制不一致。该路径可能指向一个旧版实现的遗留目录，不会被当前 Node.js 服务器使用

3.4 高扇出模块（被 ≥5 处引用）— 补充

模块	文件	扇出数	风险等级
`EnsureDocumentFocused`	`other_utils.pas`	18（通过 `Altium_API.pas` 的 `ExecuteCommand` 调用）	中
`ExecuteCommand`	`Altium_API.pas`	1（仅被 `Run` 调用），但内部通过 `case` 分发到 20+ 命令	低

分析： `EnsureDocumentFocused` 是隐式的高扇出点——它被 `ExecuteCommand` 中的每个命令入口调用，用于确保执行前正确的文档类型已聚焦。如果该函数存在 bug（如文档聚焦失败但未报错），会导致后续所有 PCB 操作返回空数据。

4. 核心模块的关键类/函数清单

4.1 MCP Server (src/index.ts)

函数/类	职责	输入	输出
`main()`	启动 MCP 服务器，连接 StdioTransport	无	`void`（启动后持续运行）
`registerLiveCommand()`	工厂函数，批量注册只读 PCB 工具	`name, title, description, command, shape, annotations`	`void`（注册到 `McpServer`）
Feature Flag 常量	控制写入工具的注册	`ENABLE_SCHEMATIC_WRITES` / `ENABLE_PCB_WRITES` / `ENABLE_SYMBOL_CREATION`	`boolean`（均默认 `false`）
`if (ENABLE_SCHEMATIC_WRITES)` 块	条件注册 `edit_schematic` 工具	Schema + handler	仅当 flag 为 `true` 时注册

函数/类	职责	输入	输出
`if (ENABLE_PCB_WRITES)` 块	条件注册 5 个 PCB 写入工具	Schema + handler	仅当 flag 为 `true` 时注册
`if (ENABLE_SYMBOL_CREATION)` 块	条件注册 `create_schematic_symbol`	Schema + handler	仅当 flag 为 `true` 时注册
`get_server_status` 处理器	收集环境信息、诊断路径	`{}`	`{ workspaceRoot, requestPath, altiumExeFound, ... }`
`altium_ping` 处理器	健康检查	`{}`	`{ protocolVersion, pong: true }`
`get_workspace_projects` 处理器	列出工作区项目	`{}`	`{ projectCount, projects[], ... }`
`get_schematic_data` 处理器	原理图数据查询	`{ schematic_full_path?, include_queries? }`	`{ components, drawing_objects }` 或分桶数据
`get_all_designators` 处理器	提取位号列表	`{}`	`string[]`（位号数组）
`get_component_pins` 处理器	查询元件引脚	`{ designators: string[] }`	`{ success, result }`

4.2 AltiumBridge (src/bridge/altiumBridge.ts)

函数/类	职责	输入	输出
`AltiumBridge` 类	桥接器核心	`{ bundledScriptsDir, timeoutMs? }`	实例方法
`executeCommand()`	执行桥接命令（含锁获取、文件读写、轮询）	`command, params, options?.timeoutMs`	`Promise<BridgeResponse>`
`ensureLayout()`	确保工作区目录和脚本同步	`options?.force?`	`void`
`resolveAltiumExePath()`	解析 X2.EXE 路径（配置 → 环境变量 → 自动发现）	无	`string \

函数/类	职责	输入	输出
`launchAltiumRunScript()`	启动 Altium 进程执行脚本	`altiumExe, scriptPrjPath`	`Promise<boolean>`
`augmentBridgeFailure()`	合并 bridge 错误日志到响应	`workspaceRoot, BridgeResponse`	`BridgeResponse`

4.3 Bridge Lock (src/bridge/lock.ts)

函数/类	职责	输入	输出
`withBridgeLock()`	获取文件锁后执行回调，超时返回	`workspaceRoot, fn, options?.waitMs?`	`Promise<T>`
`removeStaleBridgeLock()`	清理崩溃残留的锁文件	`lockPath`	`void`
`LockTimeoutError` 类	锁超时异常	`message?`	异常对象

4.4 Workspace Layout (src/workspaceLayout.ts)

函数/类	职责	输入	输出
`ensureAltiumScriptBundle()`	基于指纹的增量同步	`workspaceRoot, bundledScriptsDir, options?`	`void`
`mirrorBundledScriptsToDest()`	镜像复制脚本包（增删改）	`bundledScriptsDir, dest, currentFingerprint`	`void`
`scriptProjectPath()`	拼接脚本项目路径	`workspaceRoot`	`string`

4.5 Config (src/config.ts)

函数/类	职责	输入	输出
`loadUserConfig()`	从 `~/.altium-mcp/config.json` 加载配置	无	`UserConfig`
`saveUserConfig()`	保存配置到 `config.json`	`UserConfig`	`void`

函数/类	职责	输入	输出
`resolveWorkspaceRoot()`	按优先级解析工作区路径（环境变量 → 配置 → 默认）	`UserConfig`	`string`
`resolveAltiumExe()`	按优先级解析 X2.EXE 路径（环境变量 → 配置）	`UserConfig`	`string \
`defaultWorkspaceRoot()`	返回默认工作区路径	无	`string`

4.6 AltiumPaths (src/altiumPaths.ts)

函数/类	职责	输入	输出
`discoverAltiumExe()`	在 `C:\Program Files\Altium\AD*` 中查找最新版 X2.EXE	`baseDir?`	`string \

4.7 Bundle Fingerprint (src/altiumBundleFingerprint.ts)

函数/类	职责	输入	输出
`listBundledScriptAbsPathsSorted()`	列出 bundle 目录下所有文件路径（排序）	`bundledScriptsDir`	`string[]`
`computeBundledAltiumScriptsFingerprint()`	计算所有文件的 SHA256 指纹	`bundledScriptsDir`	`string`（hex）

4.8 Tool Definitions (src/toolDefinitions.ts)

常量	职责	值
`annotationsReadOnlyLive`	只读工具注解	`{ readOnlyHint: true, destructiveHint: false, idempotentHint: true }`
`annotationsConfigWrite`	配置写入注解	`{ readOnlyHint: false, destructiveHint: false, idempotentHint: true }`

常量	职责	值
annotationsSchematicEdit	原理图编辑注解（标记为 DISABLED）	{ readOnlyHint: false, destructiveHint: true, idempotentHint: false } + 描述头 [DISABLED]
annotationsPcbWrite	PCB 写入注解（标记为 DISABLED）	{ readOnlyHint: false, destructiveHint: true, idempotentHint: false } + 描述头 [DISABLED]
annotationsNodeOnly	本地工具注解	{ readOnlyHint: true, destructiveHint: false, idempotentHint: true }
DESCRIPTION_*	22 个活动工具 + 7 个禁用工具的详细描述	字符串常量（含 Purpose、When to use、Parameters、Returns、Prerequisites、Errors）

4.9 Edit Schematic (src/tools/editSchematic.ts)

函数/常量	职责	说明
editSchematicActionEnum	定义 11 种 action 枚举	set_component_transform, place_component, add_wire, place_power_port 等
powerPortStyleSchema	电源端口样式枚举	circle, arrow, bar, wave, gnd_power, power_ground 等 10 种
editSchematicInputSchema	Zod Schema，含 superRefine 分支校验	按 action 分支差异化校验必填参数

4.10 Get Schematic Data (src/tools/getSchematicData.ts)

函数/常量	职责	说明
schematicIncludeQuerySchema	定义 15 种 include_queries 枚举	all, sheet, components, wires, buses, net_labels 等

函数/常量	职责	说明
`getSchematicDataInputSchema`	查询参数 Zod Schema	`schematic_full_path?`, `project_full_path?`, `schematic_sheet_file_name?`, `include_queries?`

4.11 DelphiScript Entry (altium-scripts/Altium_API.pas)

函数/过程	职责	输入	输出
`Run`	入口过程：读取请求 → 解析命令 → 执行 → 写入响应	无（读取 `request.json`）	无（写入 `response.json`）
Feature Flag 常量	DelphiScript 侧门控	`ENABLE_SCHEMATIC_WRITES` / `ENABLE_PCB_WRITES` / `ENABLE_SYMBOL_CREATION`	`Boolean`（均默认 `False`）
`ExecuteCommand()`	命令路由器：`case` 分发到 20+ 命令，写入命令受 Feature Flag 门控	`CommandName: String`	`String`（JSON 响应或 `WRITE_OPERATIONS_DISABLED`）
`WriteResponse()`	统一响应封装：自动检测 `ERROR:` 前缀	`Success, Data, ErrorMsg`	无（写入 `response.json`）
`InitializeFilePaths()`	初始化文件路径	无	设置全局变量 `ROOT_DIR`、`REQUEST_FILE`、`RESPONSE_FILE`
`ExtractParameter()`	从 JSON 行中提取参数键值对	`Line: String`	追加到全局 `Params`
`BridgeWriteErrorLog()`	写入错误日志	`Msg: String`	写入 `bridge_last_error.txt`

4.12 DelphiScript Json Utils (altium-scripts/json_utils.pas)

函数/过程	职责	输入	输出
`BuildJSONObject()`	构建 JSON 对象	`Pairs: TStringList, IndentLevel`	`String`（格式化 JSON）
`BuildJSONArray()`	构建 JSON 数组	`Items: TStringList, ArrayName?, IndentLevel`	`String`（格式化 JSON）
`WriteJSONToFile()`	写入 JSON 到文件并读回	`JSON: TStringList, FileName?`	`String`（文件内容）
`AddJSONProperty()`	添加字符串属性	`List, Name, Value, IsString`	`void`（添加到 List）
`AddJSONNumber()`	添加数值属性	`List, Name, Value: Double`	`void`
`AddJSONBoolean()`	添加布尔属性	`List, Name, Value: Boolean`	`void`
`JSONEscapeString()`	JSON 字符串转义	`S: String`	`String`
`TrimJSON()`	去除 JSON 值的引号和逗号	`InputStr: String`	`String`

5. 模块级调用关系架构图

```
graph TD
    subgraph "MCP Client (Trae)"
        MC["MCP Client<br/>stdio JSON-RPC"]
    end


    subgraph "Node.js MCP Server (TypeScript)"
        subgraph "Interface Layer"
            IDX["src/index.ts<br/>McpServer 实例<br/>22 active tools + Feature
                Flag gating"]
            TD["src/toolDefinitions.ts<br/>DESCRIPTION_* 常量<br/>annotations*
                注解"]
            INST["src/mcpInstructions.ts<br/>ALTIUM_MCP_INSTRUCTIONS<br/>Advis
                ory Mode 说明"]
        end


        subgraph "Bridge Layer"
            BR["src/bridge/altiumBridge.ts<br/>AltiumBridge
                类<br/>executeCommand()"]
```

```
        LK["src/bridge/lock.ts<br/>withBridgeLock()<br/>removeStaleBridgeLock()"]
        WL["src/workspaceLayout.ts<br/>ensureAltiumScriptBundle()<br/>scriptProjectPath()"]
        FP["src/altiumBundleFingerprint.ts<br/>computeBundledAltiumScriptsFingerprint()"]
        AP["src/altiumPaths.ts<br/>discoverAltiumExe()"]
    end

    subgraph "Config Layer"
        CFG["src/config.ts<br/>loadUserConfig()<br/>resolveWorkspaceRoot()"]
    end

    subgraph "Tools / Validation Layer"
        ES["src/tools/editSchematic.ts<br/>editSchematicInputSchema<br/>11 action types"]
        GS["src/tools/getSchematicData.ts<br/>getSchematicDataInputSchema<br/>15 include_queries"]
        DG["src/tools/designators.ts<br/>parseDesignatorsFromComponentData()"]
        HP["src/tools/helpers.ts<br/>jsonResult() / errResult()"]
    end

    subgraph "File Mode (Extension)"
        FM["src/fileMode.ts<br/>getFileModeCapabilities()"]
    end
end

subgraph "File System (Workspace)"
    REQ["request.json<br/>{protocolVersion, command, ...params}"]
    RSP["response.json<br/>{success, result?, error?}"]
    LCK[".bridge.lock<br/>PID 文件锁"]
    ERR["bridge_last_error.txt<br/>错误日志"]
end

subgraph "DelphiScript Execution Layer (Altium Designer)"
    subgraph "Entry"
        API["Altium_API.pas<br/>Run → ExecuteCommand → WriteResponse<br/>+ Feature Flag 门控"]
    end
    subgraph "PCB Tools"
        PCB["pcb_utils.pas<br/>GetPCBLayers / GetAllNets<br/>GetComponentPins / ..."]
    end
    subgraph "Schematic Tools"
        SU["schematic_utils.pas<br/>GetSchematicData<br/>GetLibrarySymbolReference"]
        SE["schematic_edit.pas<br/>SchematicEditMain<br/>11 actions"]
```

```
    end
    subgraph "Other Tools"
        OU["other_utils.pas<br/>GetWorkspaceOverview<br/>EnsureDocumentFoc
            used"]
    end
    subgraph "Layout Duplicator"
        LD["pcb_layout_duplicator.pas<br/>GetLayoutDuplicatorComponents<br
            />ApplyLayoutDuplicator"]
    end
    subgraph "JSON Foundation"
        JU["json_utils.pas<br/>BuildJSONObject /
            WriteJSONToFile<br/>AddJSONProperty / ..."]
    end
end


subgraph "Altium API (COM)"
    PCB_API["IPCB_Board<br/>IPCB_Component<br/>IPCB_Net / IPCB_Pad"]
    SCH_API["ISch_Document<br/>ISch_Component<br/>ISch_Wire /
        ISch_NetLabel"]
    WS_API["IWorkspace<br/>IProject / IDocument"]
end


MC -->|"stdio JSON-RPC"| IDX
IDX -->|"import"| TD
IDX -->|"import"| INST
IDX -->|"import"| BR
IDX -->|"import"| ES
IDX -->|"import"| GS
IDX -->|"import"| DG
IDX -->|"import"| HP
IDX -->|"import"| FM
BR -->|"import"| LK
BR -->|"import"| WL
BR -->|"import"| AP
BR -->|"import"| CFG
WL -->|"import"| FP


IDX -->|"write/read"| REQ
IDX -->|"write/read"| RSP
LK -->|"create/delete"| LCK
API -->|"read"| REQ
API -->|"write"| RSP
API -->|"write"| ERR


API -->|"调用"| PCB
API -->|"调用"| SU
API -->|"调用"| SE
API -->|"调用"| OU
API -->|"调用"| LD
PCB -->|"调用"| JU
```

```
    SU -->|"调用"| JU
    SE -->|"调用"| JU
    SE -->|"调用"| SU
    OU -->|"调用"| JU
    LD -->|"调用"| JU


    PCB -->|"PCBServer API"| PCB_API
    SU -->|"SchServer API"| SCH_API
    SE -->|"SchServer API"| SCH_API
    OU -->|"GetWorkspace API"| WS_API


    BR -->|"spawn X2.EXE"| API
```

线条标注说明

线条类型	含义
实线箭头	显式 import/调用关系
虚线箭头	文件系统读写
点线箭头	进程启动（跨进程边界）
粗线框	硬件/进程边界

依赖关系结构化总结

```
1. 外部依赖 (npm)
   @modelcontextprotocol/sdk ^1.29.0  ── MCP 协议实现
   zod ^3.24.2                         ── Schema 校验
   typescript ^5.8.2                   ── 编译
   vitest ^3.1.1                       ── 测试框架


2. 文件系统依赖 (行)
   ~/.altium-mcp/config.json      ── 用配置持久化
   ~/.altium-mcp/workspace/       ── 工作区根目录
     ├── request.json             ── 请求文件
     ├── response.json            ── 响应文件
     ├── .bridge.lock             ── 文件锁
     ├── bridge_last_error.txt    ── 错误日志
     ├── bridge_user_reported_error.txt  ── 用户报告错误
     └── AltiumScript/            ── 同步的脚本包
         ├── Altium_API.PrjScr    ── 脚本工程文件
         ├── *.pas                ── 脚本源文件
         └── .mcp-bundle.sha256   ── 指纹文件


3. 进程依赖 (行)
```

```
X2.EXE (Altium Designer)      ── Windows 进程，通过 cmd spawn
```

阶段二完成。 以上分析基于 `src/` 目录下全部 TypeScript 源码、`altium-scripts/` 下全部 DelphiScript 源码、`package.json` 依赖声明、`schemas/` 下的 JSON Schema 的真实代码内容。

> **注意**：部分写入操作模块（`SchematicEdit`、PCB 写入函数、`CreateSchematicSymbol`、`ApplyLayoutDuplicator`）虽然在代码中完整实现，但通过 Feature Flag 机制从工具列表中隐藏。启用时需同步修改 `src/index.ts` 和 `altium-scripts/Altium_API.pas` 中的对应常量为 `true`。

请确认或补充信息，确认后我将进行阶段三（核心业务流程与工作原理）的分析。

阶段三：核心业务流程与工作原理

1. 核心业务流程选择

选取项目中两个最核心的流程进行分析：

流程	类型	选择原因
原理图数据读取（`get_schematic_data`）	只读查询	最常用的工具，展示了完整的请求-响应数据通路，包含复杂的过滤机制和 15 种对象类型的分桶序列化
PCB 只读查询（`get_pcb_layers` + `get_component_pins`）	只读查询	典型的 PCB 聚焦查询流程，展示了文档聚焦、数据后处理（提取位号列表）、多层信息交叉验证的完整链路，是 AI 顾问模式下最常用的操作组合

> **已禁用流程**：原理图编辑（`edit_schematic`）和 PCB 写入操作当前通过 Feature Flag 机制禁用（详见阶段一第 6 节）。代码实现完整保留，启用 Feature Flag 后即可恢复。本文档第 3 节仍保留其实现分析供参考。

2. 流程一：原理图数据读取 (get_schematic_data)

2.1 完整调用链

```
MCP Client (Trae)
  │
  │  [1] 调用 get_schematic_data 工具
  │      JSON-RPC: { method: "tools/call", params: { name:
      "get_schematic_data",
  │        arguments: { schematic_full_path?, project_full_path?,
      include_queries? } } }
  │
  ▼
src/index.ts - get_schematic_data 处理器
  │
  │  [2] 接收参数，直接透传给 bridge.executeCommand("get_schematic_data", params)
  │      说明：本层不处理 get_schematic_data 的业务逻辑，仅做类型转发
  │
  ▼
src/bridge/altiumBridge.ts - AltiumBridge.executeCommand()
  │
  │  [3] ensureLayout()：检查工作区目录和脚本包同步
  │      - 读取 workspace/AltiumScript/.mcp-bundle.sha256
  │      - 与 altium-scripts/ 的指纹比对，不一致则镜像同步
  │
  │  [4] resolveAltiumExePath()：解析 X2.EXE 路径
  │      - 优先级：config.json → env ALTIUM_MCP_ALTIUM_EXE → 自动发现
  │
  │  [5] 获取脚本项目路径 scriptProjectPath()
  │      - 路径: workspace/AltiumScript/Altium_API.PrjScr
  │
  │  [6] 删除旧 response.json（如果存在）
  │
  │  [7] 写入 request.json
  │      {
  │        "protocolVersion": 1,
  │        "command": "get_schematic_data",
  │        "schematic_full_path": "...",
  │        "include_queries": ["components", "wires", "net_labels"]
  │      }
  │
  │  [8] 启动 Altium 进程
  │      spawn("X2.EXE
      -RScriptingSystem:RunScript(ProjectName="...Altium_API.PrjScr"|ProcName=
        "Altium_API>Run)")
  │
  │  [9] 500ms 循环轮询 response.json，直到超 (120s) 或文件出现
  │
  ▼
altium-scripts/Altium_API.pas - Run 过程
  │
```

```
  │  [10] InitializeFilePaths()：算 ROOT_DIR、REQUEST_FILE、RESPONSE_FILE
  │
  │  [11] 读取 request.json 到 TStringList（逐行文本）
  │
  │  [12] 解析命令：遍历行，找到 "command"：提取值 → "get_schematic_data"
  │       ExtractParameter()：逐行解析其他参数键值对
  │
  │  [13] ExecuteCommand("get_schematic_data")
  │       - EnsureDocumentFocused("get_schematic_data") → 跳过聚焦（schematic 和
     edit 不需要聚焦）
  │       - case 'get_schematic_data' of →
     ExecuteGetSchematicData(RequestData)
  │
  ▼
altium-scripts/schematic_utils.pas - GetSchematicData()
  │
  │  [14] 获取工作区：GetWorkspace → IWorkspace
  │
  │  [15] 三分支定位项目：
  │       ├─ SchematicFullPath 非空 → 遍历所有项目文档匹配 DM_FullPath
  │       ├─ ProjectFullPath 非空 → 按路径匹配项目
  │       └─ 两者都空 → 取 WS.DM_FocusedProject
  │
  │  [16] 解析 include_queries:
  │       └─ SchDataParseStringArray → SchMcpResolveIncludes
  │           ├─ 空 token → LegacyCombined = true (legacy 模式)
  │           ├─ "all" → SchMcpSetAllIncludes (15 种全部)
  │           └─ 逐个匹配 token → 填充 IncFlags
  │
  │  [17] 遍历项目的所有原理图文档 (DM_DocumentKind='SCH')
  │       └─ 对每个匹配的 .SchDoc:
  │
  │            ┌─ [18] 元件收集 (如果 LegacyCombined 或 flags 含 'components')
  │            │    └─ 迭代器 MkSet(eSchComponent)
  │            │       └─ 每个元件：提取 designator,
x/y/width/height/rotation(mils)
  │            │       └─ 内层迭代器 MkSet(eParameter): Name/Text → parameters 对象
  │            │
  │            └─ [19] 绘图对象收集
  │                 ├─ LegacyCombined: SchCollectDrawingObjectsForSheet
     (一次性全集)
  │                 └─ 分类模式：按 flags 分别调用 SchAppendFilteredDrawings
  │                    ├─ wires → eWire 迭代器
  │                    ├─ buses → eBus + eBusEntry 迭代器
  │                    ├─ net_labels → eNetLabel 迭代器
  │                    ├─ power_ports → ePowerObject 迭代器
  │                    ├─ text_labels → eLabel 迭代器
  │                    ├─ junctions → eJunction 迭代器
  │                    ├─ ports → ePort 迭代器
  │                    ├─ off_sheet_connectors → eCrossSheetConnector 迭代器
  │                    ├─ sheet_symbols → eSheetSymbol 迭代器
  │                    ├─ directives → eNoERC + eCompileMask 迭代器
```

```
  │                    ├─ figures → 11 种图形对象迭代器
  │                    └─ sheet → SchBuildSheetInfoObject（文性    ）
  │
  │  [20] 建 JSON 输出
  │       ├─ Legacy: { "components": [...], "drawing_objects": [...] }
  │       └─ Filtered: { "schematic_data_mode": "filtered",
  │            "include_queries": [...], "sheets": [...], "components": [...],
  │            "wires": [...], "buses": [...], ... }
  │
  │  [21] WriteJSONToFile(OutputLines, ROOT_DIR + 'temp_schematic_data.json')
  │       └─ SaveToFile → LoadFromFile 读回完整文本 → 返回字符串
  │
  ▼
Altium_API.pas - WriteResponse
  │
  │  [22] 检查返回字符串是否以 'ERROR:' 开头
  │       ├─ 是 → 设置 success=false, error=ERROR: 后面的内容
  │       └─ 否 → 设置 success=true, result=返回字符串
  │
  │  [23] BuildJSONObject → SaveToFile(RESPONSE_FILE)
  │
  ▼
src/bridge/altiumBridge.ts - 轮询循环
  │
  │  [24] 检测到 response.json 存在
  │  [25] 读取文件内容, JSON.parse
  │  [26] BridgeResponseSchema.safeParse() 校验响应格式
  │       ├─ 校验通过 → 如果是 success=false, 调用 augmentBridgeFailure()
  │       └─ 校验失败 → 返回 INVALID_BRIDGE_RESPONSE
  │
  ▼
src/index.ts - get_schematic_data 处理器
  │
  │  [27] 收到 BridgeResponse → jsonResult() 封装
  │
  ▼
MCP Client
  │
  │  [28] 收到 JSON-RPC 响应: { content: [{ type: "text", text: "{...}" }] }
```

2.2 各节点职责与关键逻辑

节点	文件	职责	关键逻辑	异常处理	边界条件
[1]	MCP Client	发起工具调用	用户/AI 构造参数	超时客户端断开	参数缺失由 Zod Schema 拦截

节点	文件	职责	关键逻辑	异常处理	边界条件
[2]	src/index.ts	转发参数	传参到 bridge.executeCommand()，不处理业务	意外错误由 jsonResult 包装	参数已在 Zod 层校验通过
[3]	altiumBridge.ts	环境准备	指纹比对，增量同步	脚本目录不存在则抛出异常	force 模式下强制同步
[4]	altiumBridge.ts	路径解析	三级优先级回退	undefined 时返回 AD_NOT_FOUND	路径含空格用引号包裹
[5]	altiumBridge.ts	路径拼接	固定拼接 /AltiumScript/Altium_API.PrjScr	文件不存在返回 SCRIPT_PROJECT_NOT_FOUND	确保路径分隔符正确
[6-7]	altiumBridge.ts	写入请求	删除旧响应，写入新请求 JSON	删除失败忽略	大参数 JSON 需正确序列化
[8]	altiumBridge.ts	启动进程	spawn 带 shell:true 和 detached:true	spawn 失败返回 ALT_LAUNCH_FAILED	Altium 可能已启动，此处是再次触发执行
[9]	altiumBridge.ts	轮询等待	500ms 间隔，120s 超时	超时返回 TIMEOUT	锁可能占用，需等待锁释放
[10]	Altium_API.pas	路径初始化	从脚本项目路径反推工作区根目录	路径解析失败导致文件读写错	路径字符串以 \ 结尾
[11]	Altium_API.pas	文件读取	TStringList.LoadFromFile	文件不存在则 ShowMessage 并退出	文件编码需为 UTF-8

节点	文件	职责	关键逻辑	异常处理	边界条件
[12]	Altium_API.pas	参数解析	逐行子串匹配，非完整 JSON 解析器	未找到 command 字段返回 No command specified	不支持嵌套 JSON 对象
[13]	Altium_API.pas	命令路由	case 语句分发	未知命令 ShowMessage('Error: Unknown command')	参数通过全局变量传递
[14]	schematic_utils.pas	获取工作区	GetWorkspace 全局函数	nil 返回 ERROR: NO_WORKSPACE	无打开项目时
[15]	schematic_utils.pas	项目定位	三分支匹配逻辑	各分支对应不同错误码	同时提供 schematic_full_path 和 project_full_path 时优先前者
[16]	schematic_utils.pas	过滤解析	token 小写化后匹配，all 短路清空全选	所有 token 无效回退 legacy 模式	空数组时 LegacyCombined=true
[17]	schematic_utils.pas	文档遍历	统计 DM_DocumentKind='SCH' 数量	无原理图文档返回空数组	大型项目可能有数十个原理图页
[18]	schematic_utils.pas	元件收集	ISch_Iterator 遍历，参数嵌套迭代器	空迭代器返回空数组	元件参数可能包含多行文本
[19]	schematic_utils.pas	对象收集	按 FilterKind 分 11 种迭代器	未知 FilterKind 销毁迭代器并退出	大型图纸可能有数千个对象

节点	文件	职责	关键逻辑	异常处理	边界条件
[20]	schematic_utils.pas	JSON 构建	BuildJSONObject/Array 手动拼接	内存不足时 TStringList 异常	超大图纸 JSON 可能达到数 MB
[21]	schematic_utils.pas	文件写入	SaveToFile → LoadFromFile 两段式	文件写入失败返回 {"error": ...}	临时文件未清理仅在 FileName 空时
[22]	Altium_API.pas	响应封装	检测 ERROR: 前缀自动设置 success	错误时写入 bridge_last_error.txt	非 ERROR: 开头的错误文本不会触发 success=false
[23]	Altium_API.pas	响应写入	SaveToFile(RESPONSE_FILE)	写入失败 ShowMessage	文件路径必须可写
[24-26]	altiumBridge.ts	响应读取	轮询检测文件存在，JSON.parse + Zod 校验	格式错误返回 INVALID_BRIDGE_RESPONSE	响应可能被截断或损坏
[27]	src/index.ts	响应封装	jsonResult() 统一格式	无额外处理	大数据量 JSON 可能超 MCP 消息大小限制

2.3 性能瓶颈、风险点与设计权衡

问题	类型	说明	影响
文件轮询延迟	性能瓶颈	每次 get_schematic_data 调用至少需要 500ms-2s（Altium 启动 + 脚本执行 + 文件写入）	用户体验差，不适合频繁查询

问题	类型	说明	影响
超大图纸序列化	性能瓶颈	数千个对象的手动 `TStringList` 拼接可能导致内存压力	大图纸可能返回数 MB JSON，MCP 客户端可能截断
逐行 JSON 解析	潜在风险	DelphiScript 使用子串匹配而非完整 JSON 解析器，不支持嵌套数组	`include_queries` 数组解析依赖 JSON 格式化格式（每行一个元素），如果格式变化会解析失败
Legacy 回退陷阱	设计权衡	当 `include_queries` 参数写错（如 `"wire"` 而非 `"wires"`）时，静默回退到 legacy 模式，返回 `components + drawing_objects` 而非用户期望的分桶数据	用户可能误以为过滤生效，实际获得的是全集
`TStringList` 序列化可靠性	潜在风险	`SaveToFile`/`LoadFromFile` 的 JSON 来回转换可能引入换行符不一致	极少数情况下 JSON 格式异常
`FloatToStr` 本地化问题	设计权衡	已用 `StringReplace(..., ',', '.', ...)` 处理，但各语言环境的数字格式差异可能仍是个问题	非英语 Windows 系统可能出现坐标值含逗号
无增量更新机制	设计权衡	每次查询都全量重新读取整个原理图，没有缓存机制	重复查询同一张图时浪费 Altium API 调用

3. 流程二：PCB 只读查询（get_pcb_layers + get_component_pins）

> 当前 AI 顾问模式下最常用的操作组合：聚焦 PCB → 读取层信息 → 获取所有位号 → 查询具体元件引脚。

3.1 完整调用链（get_component_pins 为例）

```
MCP Client (Trae)
  │
  │  [1] 用在 Altium 中聚焦 .PcbDoc → 调用 get_pcb_layers
  │      JSON-RPC: { method: "tools/call", params: { name: "get_pcb_layers",
     arguments: {} } }
  │
  ▼
src/index.ts - get_pcb_layers 处理器
  │
  │  [2] registerLiveCommand 注册的透传工具
  │      bridge.executeCommand("get_pcb_layers", {})
  │
  ▼
src/bridge/altiumBridge.ts - AltiumBridge.executeCommand()
  │
  │  [3] withBridgeLock()：获取文件锁
  │  [4] ensureLayout()：脚本包同步
  │  [5] resolveAltiumExePath()：解析 X2.EXE 路径
  │  [6] 写入 request.json: { protocolVersion: 1, command: "get_pcb_layers" }
  │  [7] 启动 Altium 执行脚本
  │  [8] 轮询等待 response.json（120s 超）
  │
  ▼
Altium_API.pas - ExecuteCommand("get_pcb_layers")
  │
  │  [9] EnsureDocumentFocused → 检查是否聚焦了 .PcbDoc
  │       ├─ 未聚焦 → 返回错误
  │       └─ 已聚焦 → 继续
  │  [10] case 'get_pcb_layers' → GetPCBLayers
  │
  ▼
pcb_utils.pas - GetPCBLayers()
  │
  │  [11] GetWorkspace → IWorkspace
  │  [12] 获取活动 PCB 文档
  │  [13] PCBServer.GetCurrentPCBBoard → IPCB_Board
  │  [14] 遍历 Board.LayerStack 收集层信息
  │       └─ 每层提取：name, layerId, layerKind, isSignal, copper thickness,
     dielectric constant
  │  [15] BuildJSONArray + WriteJSONToFile → response.json
  │
  ▼
```

```
AltiumBridge → 响应读取 → 返回层列表 JSON
  │
  │  [16] AI 分析层结构 → 选取一个位号 → 调用 get_component_pins
  │       arguments: { designators: ["U1"] }
  │
  ▼
src/index.ts - get_component_pins 处理器
  │
  │  [17] 内部先调用 get_all_component_data 获取全板数据
  │       bridge.executeCommand("get_all_component_data", {})
  │  [18] parseDesignatorsFromComponentData() 提取位列表
  │       → 用户选择 U1 → 再次调用 get_all_component_data
  │  [19] 从全板数据中筛选 U1 的引脚信息
  │       → pinNumber, netName, padX, padY, layer
  │
  ▼
AltiumBridge → 返回引脚
  │
  │  [20] AI 分析引脚连接 → 生成接线建议或审查报告
```

3.2 各节点职责与关键逻辑

节点	文件	职责	关键逻辑	异常处理
[1]	MCP Client	发起工具调用	用户/AI 构造参数	聚焦 PCB 前置条件由 AI 检查
[2]	`src/index.ts`	透传转发	`registerLiveCommand` 统一注册，不处理业务	意外错误由 `jsonResult` 包装
[3-8]	`altiumBridge.ts`	桥接通信	与 `get_schematic_data` 相同的桥接流程	锁超时、进程启动失败等
[9]	`Altium_API.pas`	文档聚焦检查	`EnsureDocumentFocused("get_pcb_layers")`	PCB 未聚焦返回错误
[10-14]	`pcb_utils.pas`	PCB 数据采集	`IPCB_Board.LayerStack` 遍历	无 PCB 返回空数组
[15]	`json_utils.pas`	JSON 构建	`BuildJSONArray` + `SaveToFile`	大层叠结构 JSON 可能较大

节点	文件	职责	关键逻辑	异常处理
[17-19]	src/index.ts	数据后处理	parseDesignatorsFromComponentData() 提取位号	位号不存在返回空

3.3 AI 顾问模式下的典型工作流

```
1. get_server_status     → 确认环境正常
2. get_pcb_layers        → 了解层结构
3. get_pcb_layer_stackup → 了解层叠参数
4. get_all_nets          → 获取网络列表
5. get_all_designators   → 获取元件清单
6. get_component_pins    → 查询关键元件引脚
7. get_schematic_data    → 交叉验证原理图设计
8. 综合分析 → 生成审查报告 / 选型建议 / 优化方案
```

4. 流程三：原理图编辑 (edit_schematic) — 已禁用

> ⚠ **当前状态**：edit_schematic 通过 ENABLE_SCHEMATIC_WRITES = false Feature Flag 禁用。
>
> 调用时返回 WRITE_OPERATIONS_DISABLED 错误。代码实现完整保留在 schematic_edit.pas 中。
>
> **启用方法**：将 src/index.ts 和 altium-scripts/Altium_API.pas 中的 ENABLE_SCHEMATIC_WRITES 改为 true，重新编译后即可恢复。

以下为该流程的实现分析（供启用后参考）：

4.1 完整调用链

以 place_component action 为例（最复杂的 action）：

```
MCP Client
  |
  |  [1] 调用 edit_schematic 工具
  |      JSON-RPC: { method: "tools/call", params: { name: "edit_schematic",
  |        arguments: {
  |          action: "place_component",
  |          schematic_full_path: "D:/Project/Sheet1.SchDoc",
  |          lib_reference: "Res2",
```

```
  │          designator: "R1",
  │          x_mils: 5000, y_mils: 3000,
  │          rotation_deg: 0,
  │          sch_library_path: "C:/Library/Miscellaneous Devices.SchLib"
  │        } } }
  │
  ▼
src/tools/editSchematic.ts - Zod Schema 校验
  │
  │  [2] action="place_component" → 进入 place_component 校验分支
  │      ├─ 校验 lib_reference 非空              ✓ "Res2" 通过
  │      ├─ 校验 designator 非空                ✓ "R1" 通过
  │      ├─ 校验 x_mils 和 y_mils 非 undefined   ✓ 通过
  │      └─ 校验 sheet 定位参数非空              ✓ schematic_full_path 提供
  │
  │  [3] 校验通过，进入处理器
  │
  ▼
src/index.ts - edit_schematic 处理器
  │
  │  [4] 调用 bridge.executeCommand("schematic_edit", params, { timeoutMs:
  │   240_000 })
  │      注意：编辑操作超时 240s（默认 120s 的两倍），因为 Altium 图形界面
  │      操作可能较慢，且锁等待时间也计入超时
  │
  ▼
src/bridge/altiumBridge.ts - executeCommand()
  │
  │  [5] withBridgeLock()：获取文件锁 .bridge.lock（默认 60s 等待）
  │      ├─ 锁获取成功 → 继续
  │      └─ 锁超时 → LockTimeoutError
  │
  │  [6] ensureLayout() → 同步脚本
  │  [7] resolveAltiumExePath() → 获取 X2.EXE 路径
  │  [8] 删除旧 response.json，写入 request.json
  │  [9] launchAltiumRunScript() → 启动 Altium 执行脚本
  │  [10] 轮询等待 response.json（240s 超，500ms 间隔）
  │
  ▼
Altium_API.pas - Run 过程
  │
  │  [11] 读取 request.json，解析 command="schematic_edit"
  │  [12] ExecuteCommand("schematic_edit") → SchematicEditMain(RequestData)
  │
  ▼
altium-scripts/schematic_edit.pas - SchematicEditMain()
  │
  │  [13] 解析 action="place_component"（归一化，无 alias 转换）
  │  [14] 解析 sheet 定位参数：
  │       ├─ "schematic_full_path" → "D:/Project/Sheet1.SchDoc"
  │       └─ "project_full_path" 未提供
  │
```

```
  │  [15] SchEditResolveSheet()：定位原理图文档
  │       └─ 遍历 workspace 项目，匹配 DM_FullPath
  │       └─ Client.OpenDocument('SCH', ...) → 打开文档
  │       └─ SchServer.GetSchDocumentByPath → ISch_Document
  │       └─ 失败返回 ERROR: SCHEMATIC_NOT_IN_OPEN_PROJECTS 等
  │
  │  [16] dispatcher: action = "place_component"
  │       └─ 解析 action 专属参数：
  │           ├─ SchEditParseStringAfterKey → "lib_reference" → "Res2"
  │           ├─ SchEditParseStringAfterKey → "designator" → "R1"
  │           ├─ SchEditParseStringAfterKey → "sch_library_path" →
     "C:/Library/..."
  │           ├─ SchEditParseFloatAfterKey → "x_mils" → 5000.0
  │           ├─ SchEditParseFloatAfterKey → "y_mils" → 3000.0
  │           └─ SchEditParseFloatAfterKey → "rotation_deg" → 0.0
  │
  │  [17] 调用 SchEditPlaceComponent(SchDoc, "Res2", "R1", 5000, 3000, 0,
     LibPath)
  │
  ▼
schematic_edit.pas - SchEditPlaceComponent()
  │
  │  [18] 检查 sch_library_path 是否提供
  │       ├─ 提供 → 分支 A：库放置
  │       │   ├─ FileExists(LibPath) 检查 → 不存在返回 ERROR: SCH_LIBRARY_NOT_FOUND
  │       │   ├─ SchDoc.PlaceSchComponent(LibPath, LibRef) 放置
  │       │   │   └─ 返回放置后的元件对象
  │       │   └─ 放置失败返回 ERROR: PLACE_SCH_COMPONENT_FAILED
  │       │
  │       └─ 未提供 → 分支 B：空白放置
  │           └─ SchObjectFactory(eSchComponent) 创建空白元件
  │               └─ 设置 LibReference, Designator, 位置, 旋转
  │               └─ 注册到文档容器
  │
  │  [19] 设置元件属性：
  │       ├─ SchComponent.LibReference := "Res2"
  │       ├─ SchComponent.Designator.Name := "R1"
  │       ├─ SchComponent.Location := Point(MilsToCoord(5000),
     MilsToCoord(3000))
  │       └─ SchComponent.Orientation := SchEditRotationFromDeg(0)
  │
  │  [20] 注册到原理图文档
  │       └─ SchDoc.AddSchObject(SchComponent)
  │       └─ SchServer.RobotManager.SendMessage(...) 通知 UI 更新
  │
  │  [21] 返回放置情 JSON: { "designator": "R1", "lib_reference": "Res2",
  │         "x_mils": 5000, "y_mils": 3000, "rotation_deg": 0,
     "sch_library_path": "..." }
  │
  ▼
SchematicEditMain() - 响应包装
  │
```

```
  │  [22] 检查 Inner 是否以 "ERROR:" 开头
  │       └─ 否 → 构建成功响应
  │           { "action": "place_component", "sheet": "D:/.../Sheet1.SchDoc",
  │             "details": { "designator": "R1", ... } }
  │
  ▼
Altium_API.pas - WriteResponse → SaveToFile(RESPONSE_FILE)
  │
  ▼
src/bridge/altiumBridge.ts - 轮询返回
  │
  │  [23] 读取 response.json，校验格式，返回
  │
  ▼
src/index.ts - edit_schematic 处理器
  │
  │  [24] jsonResult() 封装 → 返回给 MCP Client
  │
  ▼
MCP Client
  │
  │  [25] 收到响应，AI 分析结果
  │       建议：调用 get_schematic_data 验证放置结果
```

4.2 各节点职责与关键逻辑（编辑模式特有）

节点	文件	职责	关键逻辑	异常处理	边界条件
[2]	editSchematic.ts	分支校验	superRefine 按 action 差异化校验	校验失败返回 Zod 错误信息	11 种 action 各有不同的必填参数组合
[4]	src/index.ts	设置长超时	timeoutMs: 240_000	超时返回 TIMEOUT	编辑操作通常比查询慢 2-10 倍
[5]	lock.ts	文件锁互斥	wx 标志创建独占锁文件	锁超时 LockTimeoutError	并发编辑调用会串行化
[15]	schematic_edit.pas	文档定位	与 get_schematic_data 共享相似逻辑	7 种不同错误码	文档必须已打开在 workspace 中

节点	文件	职责	关键逻辑	异常处理	边界条件
[16]	schematic_edit.pas	参数解析	使用 SchEditParse* 系列函数，区分"未提供"和"提供 0"	缺失必填参数立即返回 ERROR: XXX_REQUIRED	通过 var Found 标志位区分未提供 vs 0 值
[17]	schematic_edit.pas	库放置	两分支逻辑	SCH_LIBRARY_NOT_FOUND / PLACE_SCH_COMPONENT_FAILED	库路径必须绝对路径，Altium 不支持相对路径
[18]	schematic_edit.pas	空白元件创建	SchObjectFactory 创建，手动设置属性	FACTORY_SCH_COMPONENT_FAILED	无图形，仅占位符
[19]	schematic_edit.pas	坐标转换	MilsToCoord 将 mils 转为内部坐标	类型转换异常被外层 try/except 捕获	坐标值需在图纸范围内
[20]	schematic_edit.pas	文档注册	AddSchObject + RobotManager 消息	注册失败可能导致文档不一致	不自动保存文档
[21]	schematic_edit.pas	结果 JSON	返回放置元件的详细信息	无额外处理	sch_library_path 回显到响应
[22]	schematic_edit.pas	错误检查	检测 ERROR: 前缀	透传错误码，跳过 JSON 包装	外层 try/except 兜底 SCHEMATIC_EDIT_EXCEPTION

4.3 性能瓶颈、风险点与设计权衡

问题	类型	说明	影响
串行化执行	性能瓶颈	文件锁 `.bridge.lock` 强制所有命令串行执行，即使编辑和查询互不冲突	编辑操作期间，所有查询操作必须等待
无事务回滚	潜在风险	`edit_schematic` 操作不提供撤销/回滚机制，如果编辑中间出错，Altium 文档可能处于不一致状态	破坏性操作后需手动在 Altium 中保存或撤销
不自动保存	设计权衡	编辑操作不主动保存 .SchDoc，依赖用户在 Altium 中手动保存或 Altium 自动保存	意外崩溃导致编辑丢失；但避免了自动保存导致不可撤销的修改
库路径依赖	边界条件	`place_component` 依赖 `sch_library_path` 提供绝对路径，Altium 库搜索路径规则复杂	用户必须知道库文件的完整路径，否则只能放置空白占位符
无锁超时时的阻塞	潜在风险	如果 Altium 脚本因断点调试而挂起，文件锁不会释放，后续所有工具调用都会在 60s 锁等待后超时	需要通过 `removeStaleBridgeLock` 手动清理
坐标单位一致性	设计权衡	所有坐标以 mils 为单位，Altium 内部使用 1/10000 mil 的单位	转换可能引入取整误差，但 mils 对于 PCB 设计是标准单位

问题	类型	说明	影响
无并发控制	设计权衡	没有对 Altium 侧编辑状态的并发控制，两个独立的 MCP 客户端可能同时编辑同一张原理图	最后一个写入 `request.json` 的客户端会覆盖前一个请求

4.4 编辑流程的 11 种 action 实现复杂度对比

Action	实现复杂度	参数数量	关键风险
`set_component_transform`	低	4（+3 可选标志位）	坐标超出图纸边界
`set_component_parameters`	中	3（含数组）	数组长度不匹配
`place_component`	高	6	库路径不存在导致空白放置
`add_text`	低	3	文本编码问题
`add_net_label`	低	4	网络名已存在（重复创建）
`add_wire`	中	1（CSV 串）	CSV 格式错误，线段数量过多
`add_bus`	中	1（CSV 串）	同上
`add_bus_entry`	低	1（CSV 串）	必须恰好 4 个数字
`place_power_port`	中	5	电源样式枚举值错误
`place_gnd`	低	3	默认样式不匹配预期
`place_vcc`	低	3	同上

5. 核心业务流程时序图

5.1 原理图数据读取时序图

```
sequenceDiagram
    participant Client as MCP Client
    participant Index as src/index.ts
```

```
participant Bridge as src/bridge/altiumBridge.ts
participant Lock as src/bridge/lock.ts
participant FS as 文件系 (Workspace)
participant AD as Altium Designer
participant API as Altium_API.pas
participant SU as schematic_utils.pas
participant JU as json_utils.pas


Client->>Index: 调用 get_schematic_data(args)
Note over Index: Zod Schema 校验通过


Index->>Bridge: executeCommand("get_schematic_data", args)


Bridge->>Lock: withBridgeLock(workspaceRoot)
Note over Lock: 尝试获取 .bridge.lock 文件锁


alt 锁获取成功
    Lock-->>Bridge: 锁获取成功
    Bridge->>FS: 删除旧 response.json
    Bridge->>FS: 写入 request.json
    Note over FS: { protocolVersion:1, command:"get_schematic_data", ... }


    Bridge->>AD: spawn X2.EXE -RScriptingSystem:RunScript(...)
    Note over AD: 异步启动，不等待进程退出


    loop 每 500ms 轮询，最长 120s
        Bridge->>FS: 检查 response.json 是否存在?
        alt 文件存在
            Bridge->>FS: 读取 response.json
            Bridge->>Bridge: JSON.parse + Zod Schema 校验
            Note over Bridge: augmentBridgeFailure 合并错误日志
            Bridge-->>Index: 返回 BridgeResponse
            Index-->>Client: jsonResult 封装
        else 文件不存在
            Bridge->>Bridge: 继续等待
        end
    end


    Bridge->>Bridge: 超 → 返回 TIMEOUT
    Bridge-->>Index: 返回错误
    Index-->>Client: 返回错误 JSON


else 锁超时
    Lock-->>Bridge: LockTimeoutError
    Bridge-->>Index: 返回错误
    Index-->>Client: 返回错误
end
```

```
    Note over AD: ---- Altium Designer 内部流程 ----


    AD->>API: 加载脚本项目 Altium_API.PrjScr
    API->>API: InitializeFilePaths()
    API->>FS: 读取 request.json


    alt 文件存在
        API->>API: 解析 command="get_schematic_data"
        API->>API: ExecuteCommand("get_schematic_data")
        API->>API: EnsureDocumentFocused → 跳过


        API->>SU: ExecuteGetSchematicData(RequestData)
        SU->>SU: GetSchematicData(ROOT_DIR, ...)
        SU->>SU: GetWorkspace → IWorkspace


        alt 项目定位失败
            SU-->>API: 返回 "ERROR: NO_WORKSPACE" 等
        else 项目定位成功
            SU->>SU: 解析 include_queries 过滤
            SU->>SU: 遍历原理图文档
            SU->>SU: 迭代器收集元件 + 绘图对象
            SU->>JU: BuildJSONArray / BuildJSONObject
            JU-->>SU: JSON 字符串
            SU->>JU: WriteJSONToFile → temp_schematic_data.json
            JU-->>SU: 返回 JSON 文本
            SU-->>API: 返回 JSON 字符串
        end


        API->>API: WriteResponse(success, result, error)
        Note over API: 检测 "ERROR:" 前缀自动设置 success=false
        API->>JU: BuildJSONObject + SaveToFile(response.json)
        JU-->>FS: 写入 response.json
    else 文件不存在
        API->>API: ShowMessage + Exit
    end
```

5.2 原理图编辑时序图（place_component 为例）— 供启用后参考

> ⚠ 当前 `edit_schematic` 通过 Feature Flag 禁用，以下时序图供参考。

```
sequenceDiagram
    participant Client as MCP Client
    participant Zod as src/tools/editSchematic.ts
    participant Index as src/index.ts
    participant Bridge as src/bridge/altiumBridge.ts
    participant Lock as src/bridge/lock.ts
```

```
participant FS as 文件系
participant AD as Altium Designer
participant API as Altium_API.pas
participant SE as schematic_edit.pas
participant JU as json_utils.pas


Client->>Zod: 调用 edit_schematic({ action: "place_component", ... })


alt Zod 校验失败
    Zod-->>Client: Zod 错误信息
else 校验通过
    Zod-->>Index: 校验通过


    Index->>Bridge: executeCommand("schematic_edit", params, { timeoutMs:
        240000 })
    Note over Bridge: 编辑操作超时设为 240s（默认 2 倍）


    Bridge->>Lock: withBridgeLock(workspaceRoot, { waitMs: 60000 })
    Note over Lock: 锁等待 60s，50ms 轮询


    alt 锁获取失败
        Lock-->>Bridge: LockTimeoutError
        Bridge-->>Index: 返回错误
        Index-->>Client: 返回错误
    else 锁获取成功
        Lock-->>Bridge: 锁获取成功
        Bridge->>FS: 删除旧 response.json
        Bridge->>FS: 写入 request.json
        Note over FS: { protocolVersion:1, command:"schematic_edit",
            action:"place_component", ... }


        Bridge->>AD: spawn X2.EXE -RScriptingSystem:RunScript(...)


        loop 每 500ms 轮询，最长 240s
            Bridge->>FS: 检查 response.json?
            alt 已存在
                Bridge->>FS: 读取 + 校验
                Bridge-->>Index: BridgeResponse
                Index-->>Client: jsonResult
            else 不存在
                Bridge->>Bridge: 继续等待
            end
        end
        Bridge-->>Index: TIMEOUT
        Index-->>Client: 错误
    end
end
```

```
Note over AD: ---- Altium 内部编辑流程 ----

AD->>API: 加载脚本
API->>FS: 读取 request.json
API->>API: 解析 command="schematic_edit"
API->>API: ExecuteCommand("schematic_edit")
API->>API: EnsureDocumentFocused → 跳过
API->>SE: SchematicEditMain(RequestData)

SE->>SE: 解析 action="place_component"
SE->>SE: 解析 sheet 定位参数
SE->>SE: SchEditResolveSheet(...)

alt 文档定位失败
    SE-->>API: 返回 ERROR: SCHEMATIC_NOT_IN_OPEN_PROJECTS 等
    API->>FS: 写入 response.json (success=false)
else 文档定位成功
    SE->>SE: 解析 action 专属参数
    Note over SE: lib_reference, designator, x_mils, y_mils,
        sch_library_path

    SE->>SE: SchEditPlaceComponent(SchDoc, "Res2", "R1", 5000, 3000, 0,
        LibPath)

    alt sch_library_path 提供
        alt 库文件不存在
            SE-->>SE: 返回 ERROR: SCH_LIBRARY_NOT_FOUND
        else 库放置失败
            SE-->>SE: 返回 ERROR: PLACE_SCH_COMPONENT_FAILED
        else 放置成功
            Note over SE: PlaceSchComponent 完成
            SE->>SE: 设置位置/旋转
            SE->>SE: 注册到文档
        end
    else 未提供库路径
        SE->>SE: SchObjectFactory 创建空白元件
        SE->>SE: 手动设置属性
        SE->>SE: AddSchObject 注册
    end

    alt 操作成功
        SE->>SE: 构建详情 JSON
        SE->>JU: BuildJSONObject
        JU-->>SE: 成功 JSON
        SE-->>API: 返回 action+sheet+details JSON
    else 操作失败 (ERROR: 前缀)
        SE-->>API: 返回 ERROR: 错误字符串
```

```
        end
    end


    alt 异常捕获
        API->>API: WriteResponse 检测 ERROR: 前缀
        API->>FS: 写入 bridge_last_error.txt
        API->>JU: BuildJSONObject + SaveToFile(response.json)
        JU-->>FS: 写入 response.json
    else 全局异常
        SE->>SE: try/except 捕获 → ERROR: SCHEMATIC_EDIT_EXCEPTION
        API->>FS: 写入 response.json
    end
```

6. 项目工作原理总结

6.1 各模块协作完成核心业务

altium-mcp 的运作建立在三个核心机制的协作之上：

机制一：文件桥接通信（跨语言 IPC）

Node.js 与 DelphiScript 之间通过文件系统传递 JSON 消息。MCP 服务器写入 `request.json`，启动 Altium 脚本执行，然后轮询等待 `response.json`。这种机制看似简单，但有效解决了两个关键问题：

1. **进程隔离**：Node.js 和 Altium 运行在独立的进程中，任何一方崩溃都不会影响另一方。Altium 的 DelphiScript 崩溃只会导致 `response.json` 未写入，Node.js 侧会因超时而返回错误，不会挂掉 MCP 服务器。
2. **零依赖**：不需要安装额外的序列化库、网络库或中间件。Altium 的 DelphiScript 环境对网络 API 支持有限，但文件读写是原生支持的。

机制二：四层架构的解耦

请求从 MCP Client 到 Altium API 经历了四个明确的层次：

层	职责	关键设计
接口层	工具注册、参数描述、转发	Zod Schema 使用 `superRefine` 实现差异化校验，`registerLiveCommand` 工厂函数批量注册

层	职责	关键设计
桥接层	文件通信、锁管理、进程管理	`withBridgeLock` 使用 PID 文件锁 + stale lock 自动清理，确保并发安全
执行层	业务逻辑、Altium API 调用	`ExecuteCommand` 用 `case` 分发 20+ 命令，统一的 WebResponse 封装
基础层	JSON 序列化、错误日志	`json_utils.pas` 手动实现 JSON 构建，轻量无依赖

机制三：脚本包增量同步

`ensureAltiumScriptBundle()` 使用 SHA256 指纹确保 workspace 中的脚本包与 `altium-scripts/` 目录保持同步。只有内容变化时才重新复制，避免每次启动都覆盖文件。同步采用镜像策略（增删改），而非删除重建，确保 Altium 对脚本项目路径的稳定引用。

6.2 设计上的巧妙之处

1. 统一的错误传播协议

DelphiScript 定义了 `ERROR: XXX` 的错误码传播协议，贯穿整个执行层：

- 底层函数返回以 `ERROR:` 为前缀的字符串
- 中间层函数检测 `ERROR:` 前缀后直接透传，跳过 JSON 包装
- 顶层 `WriteResponse` 自动检测 `ERROR:` 前缀，设置 `success=false` 并提取错误信息
- Node.js 侧的 `augmentBridgeFailure` 进一步合并 Altium 写入的 `bridge_last_error.txt` 到错误信息中

这种设计使得错误信息从 Altium API 一路传递到 MCP Client，没有信息丢失。

2. 文件锁的 stale lock 自动清理

`removeStaleBridgeLock()` 通过 `process.kill(pid, 0)` 检测锁文件中的 PID 是否存活。如果进程已崩溃（MCP 服务器被杀掉或 Altium 异常退出），锁文件会被自动清理，避免死锁。这是典型的"自愈"设计，对于桌面应用至关重要。

3. 双模式原理图数据查询

`get_schematic_data` 支持 legacy 模式（`components + drawing_objects` 全集）和 filtered 模式（15 种分桶过滤）。向后兼容 legacy 模式的同时，通过 `include_queries` 参数提供了精细化的数据过滤能力，避免大型原理图的数据量淹没 AI 的上下文窗口。

4. 编辑操作的多 action 策略模式

`edit_schematic` 通过单一工具入口分发 11 种不同的编辑操作，每种操作有独立的参数校验规则。Zod 的 `superRefine` 实现了 TypeScript 层的差异化校验，而 DelphiScript 的 `SchematicEditMain` 实现了执行层的分发。新增 action 只需添加枚举值和对应分支，无需修改现有代码。

5. 电源端口的便捷别名

`place_gnd` 和 `place_vcc` 作为 `place_power_port` 的快捷方式，简化了最常见的两种电源端口放置操作。用户无需指定 `power_port_style` 和 `net_name`，系统自动使用默认值（GND 样式 + "GND" 网络名，Arrow 样式 + "VCC" 网络名）。这是典型的"约定优于配置"设计。

6.3 架构决策的权衡回顾

决策	选择	放弃	合理性
进程间通信方式	JSON 文件桥接	Socket / Named Pipe / HTTP	受 DelphiScript 环境限制，文件 I/O 是唯一可靠且跨版本兼容的方案
编辑操作保存策略	不自动保存	自动保存 + 撤销	避免不可逆修改，但增加用户操作步骤

决策	选择	放弃	合理性
并发控制	单文件锁串行化	多读单写锁 / 无锁	串行化简单可靠，但并发性能差。Altium 本身也不支持多实例并行操作
JSON 库选择	手动 TStringList 拼接	正式 JSON 库（如 JSONObject）	减少依赖，但 JSON 格式鲁棒性依赖手动维护
脚本同步方式	增量镜像同步	删除重建 / 版本控制	保持 Altium 路径稳定，避免频繁重建脚本项目
写入操作策略	Feature Flag 门控（默认禁用）	直接暴露 / 独立进程	解决写入操作稳定性问题，保留代码实现便于后续启用

阶段三完成。 以上分析基于 `src/` 目录下全部 TypeScript 源码、`altium-scripts/` 下全部 DelphiScript 源码、`schemas/` 下的 JSON Schema 的真实代码内容。

项目解析报告总览

阶段	内容	核心发现
阶段一	全局架构与核心设计思路	分层 + 文件桥接代理模式，4 层架构，5 种设计模式，Feature Flag 三层门控架构
阶段二	模块依赖与文件调用关系	14 个 Node.js 模块 + 7 个 DelphiScript 模块，`json_utils.pas` 和 `AltiumBridge` 为高扇出风险节点

阶段	内容	核心发现
阶段三	核心业务流程与工作原理	3 个核心流程的完整调用链（原理图读取、PCB 只读查询、原理图编辑[已禁用]），2 幅时序图，性能瓶颈与设计权衡分析